

Linguagens e Ambientes de Programação (Aula Teórica 14)

LEI - Licenciatura em Engenharia Informática

João Costa Seco (joao.seco@fct.unl.pt)



NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

Agenda

- Sistemas de módulos
- Módulos OCaml
- Functores em OCaml, Traits em Scala e RUST
- Mutabilidade

Sistema de Módulos

- Espaços de nomes
 - Grupos de declarações (normalmente) relacionadas isolados de outros grupos por via da qualificação dos nomes num módulo.
 - Permite a reutilização dos mesmos nomes em contextos diferentes sem colisões.
 - Pacotes e classes em Java, ficheiros/módulos em C, estruturas/módulos em ocaml

C

```
#include "stack.h"
// int push(int x);

#include "queue.h"
// int push(int x);

/* CONFLICT – duplicate definition */
```

Java

```
import java.util.Date;
import java.sql.Date;

java.util.Date now = new java.util.Date();
java.sql.Date sqlDate = new java.sql.Date(now.getTime());
```


Sistema de Módulos

- Espaços de nomes
 - Grupos de declarações (normalmente) relacionados isolados de outros grupos por via da qualificação dos nomes num módulo.
 - Permite a reutilização dos mesmos nomes em contextos diferentes sem colisões.
 - Pacotes e classes em Java, ficheiros/módulos em C, estruturas/módulos em ocaml
- Abstração
 - Permite esconder/revelar selectivamente informação (*information hiding*)
 - Declarações em ficheiros “header” C e interfaces em Java.
 - Isolamento de código, melhor desenvolvimento e manutenção, ownership, etc.

Java

```
Map<Integer, String> hm = new HashMap<>();  
Map<Integer, String> ht = new TreeMap<>();  
hm.put(1, "one");  
hm.put(2, "two");  
hm.put(3, "three");  
ht.put(1, "one");  
ht.put(2, "two");  
ht.put(3, "three");
```

Sistema de Módulos

- Espaços de nomes
 - Grupos de declarações (normalmente) relacionados isolados de outros grupos por via da qualificação dos nomes num módulo.
 - Permite a reutilização dos mesmos nomes em contextos diferentes sem colisões.
 - Pacotes e classes em Java, ficheiros/módulos em C, estruturas/módulos em ocaml
- Abstração
 - Permite esconder/revelar selectivamente informação (*information hiding*)
 - Isolamento de código, melhor desenvolvimento e manutenção, ownership, etc.
- Reutilização de código
 - reutilização sem cópia, modularidade, (cf. herança em Java)

Sistema de Módulos

- Espaços de nomes
 - Grupos de declarações (normalmente) relacionadas isolados de outros grupos por via da qualificação dos nomes num módulo.
 - Permite a reutilização dos mesmos nomes em contextos diferentes sem colisões.
 - Pacotes e classes em Java, ficheiros/módulos em C, estruturas/módulos em ocaml
- Abstração
 - Permite esconder/revelar selectivamente informação (*information hiding*)
 - Isolamento de código, melhor desenvolvimento e manutenção, ownership, etc.
- Reutilização de código
 - reutilização sem cópia, modularidade, (cf. herança em Java)
- (em OCaml) Parametrização de módulos, (em Scala, Rust) Traits
 - Os Functores em OCaml são como funções de módulos para módulos (cf. traits em Scala ou Rust)

Sistema de Módulos

- Espaços de nomes

- Grupos de declarações

- Permite a reutilização

- Pacotes e classes

- Abstração

- Permite esconder

- Isolamento de código

- Reutilização de código

- reutilização sem

- (em OCaml) Polimorfismo

- Os Functores em

nomes num módulo.

```
trait Map[K, V] {
  def put(key: K, value: V): Unit
  def get(key: K): Option[V]
}

class HashMap[K, V] extends Map[K, V] {
  private val data = scala.collection.mutable.HashMap[K, V]()
  def put(key: K, value: V): Unit = data(key) = value
  def get(key: K): Option[V] = data.get(key)
}

class TreeMap[K, V](implicit ord: Ordering[K]) extends Map[K, V] {
  private val data = scala.collection.mutable.TreeMap[K, V]()
  def put(key: K, value: V): Unit = data(key) = value
  def get(key: K): Option[V] = data.get(key)
}

val m1: Map[Int, String] = new HashMap()
m1.put(1, "one")
println(m1.get(1))

val m2: Map[Int, String] = new TreeMap()
m2.put(1, "one")
println(m2.get(1))
```


Sistema de Módulos

- Es
 - G
 - P
 - P
 - Ab
 - P
 - Is
 - Re
 - re
 - (em
 - ○
- ```
trait Printable[K, V] { self: Map[K, V] =>
 def prettyPrint(): Unit = {
 println(s"[${this.getClass.getSimpleName}]")
 println(this.toString)
 }
}

class HashMap[K, V] extends scala.collection.mutable.HashMap[K, V]
 with Printable[K, V]

class TreeMap[K, V](implicit ord: Ordering[K]) extends scala.collection.mutable.TreeMap[K, V]
 with Printable[K, V]

val hm = new HashMap[Int, String]()
hm.put(1, "one")
hm.put(2, "two")

val tm = new TreeMap[Int, String]()
tm.put(1, "one")
tm.put(2, "two")

hm.prettyPrint()
tm.prettyPrint()
```



# Sistema de Módulos

- Espaços de nomes
  - Grupos de declarações (normalmente) num ficheiro ou es num módulo.
  - Permite a reutilização dos mesmos nomes
  - Pacotes e classes em Java, ficheiros/módulos em Python
- Abstracção
  - Permite esconder/revelar selectivamente partes do código
  - Isolamento de código, melhor desenvolvimento
- Reutilização de código
  - reutilização sem cópia, modularidade, (compilação)
- (em OCaml) Parametrização de módulos
  - Os Functores em OCaml são como funções

```
use std::collections::{HashMap, BTreeMap};
use std::any::type_name;

// A flexible extension to a struct that implements function "entries"
trait Printable {
 fn entries(&self) -> Vec<(i32, String)>;

 fn pretty_print(&self) {
 for (k, v) in self.entries() {
 println!(" {} => {}", k, v);
 }
 }
}

// Extension of HashMap with Printable trait
impl Printable for HashMap<i32, String> {
 fn entries(&self) -> Vec<(i32, String)> {
 self.iter().map(|(k, v)| (*k, v.clone())).collect()
 }
}

// Extension of BTreeMap with Printable trait
impl Printable for BTreeMap<i32, String> {
 fn entries(&self) -> Vec<(i32, String)> {
 self.iter().map(|(k, v)| (*k, v.clone())).collect()
 }
}

fn main() {
 let mut hm = HashMap::new();
 hm.insert(1, "one".to_string());
 hm.insert(2, "two".to_string());

 let mut bt = BTreeMap::new();
 bt.insert(1, "one".to_string());
 bt.insert(2, "two".to_string());

 hm.pretty_print();
 bt.pretty_print();
}
```

es num módulo.

# Módulos em OCaml

# Módulos em OCaml

- Os módulos são definidos por estruturas (**struct**)
- Os tipos para os módulos são assinaturas (**sig**)
- As definições de tipos por omissão são públicas (**type**)
- As implementações dos nomes ficam privadas (**val**)

```
module MyList = struct
 type 'a list = Nil | Cons of 'a * 'a list
 let empty = Nil
 let rec length = function
 | Nil → 0
 | Cons (_, xs) → 1 + length xs
 let insert x xs = Cons (x, xs)
 let head = function
 | Nil → None
 | Cons (x, _) → Some x
 let tail = function
 | Nil → None
 | Cons (_, xs) → Some xs
end
```

[7] ✓ 0.0s

```
... module MyList :
 sig
 type 'a list = Nil | Cons of 'a * 'a list
 val empty : 'a list
 val length : 'a list → int
 val insert : 'a → 'a list → 'a list
 val head : 'a list → 'a option
 val tail : 'a list → 'a list option
 end
```



# Espaço de nomes

- Os nomes declarados num módulo podem ser usados de forma qualificada (com o nome do Módulo e um ponto: **List.fold\_right**)
- Ou pode-se usar a directiva **open** para expandir os nomes do módulo usado no módulo cliente.
- o módulo **StdLib** está sempre aberto.

```
module MyStack = struct
 type 'a stack = 'a MyList.list
 let empty = MyList.empty
 let push x xs = MyList.insert x xs
 let pop = MyList.tail
 let top = MyList.head
end

[17] ✓ 0.0s

... module MyStack :
 sig
 type 'a stack = 'a MyList.list
 val empty : 'a MyList.list
 val push : 'a → 'a MyList.list → 'a MyList.list
 val pop : 'a MyList.list → 'a MyList.list option
 val top : 'a MyList.list → 'a option
 end
```

```
module MyStack = struct
 open MyList

 type 'a stack = 'a list
 let empty = empty
 let push x xs = insert x xs
 let pop = tail
 let top = head
end

[14] ✓ 0.0s

... module MyStack :
 sig
 type 'a stack = 'a MyList.list
 val empty : 'a MyList.list
 val push : 'a → 'a MyList.list → 'a MyList.list
 val pop : 'a MyList.list → 'a MyList.list option
 val top : 'a MyList.list → 'a option
 end
```



# Abstração de nomes

- Os tipos dos módulos permitem ainda esconder a definição dos tipos
- Uma assinatura pode ter várias implementações compatíveis (opacas)

```
module type Stack =
 sig
 type 'a stack
 val empty : 'a stack
 val push : 'a → 'a stack → 'a stack
 val pop : 'a stack → 'a stack option
 val top : 'a stack → 'a option
 end

module MyStack : Stack

module AnotherStack : Stack
```

```
module type Stack = sig
 type 'a stack
 val empty : 'a stack
 val push : 'a → 'a stack → 'a stack
 val pop : 'a stack → 'a stack option
 val top : 'a stack → 'a option
end
```

```
module MyStack:Stack = struct
 type 'a stack = 'a MyList.list
 let empty = MyList.empty
 let push x xs = MyList.insert x xs
 let pop = MyList.tail
 let top = MyList.head
end
```

```
module AnotherStack : Stack = struct
 type 'a stack = 'a list
 let empty = []
 let push x xs = x :: xs
 let pop = function
 | [] → None
 | _ :: xs → Some xs
 let top = function
 | [] → None
 | x :: _ → Some x
end
```

# Tipos e nomes

- A especialização de tipos de módulos pode ser feita com um módulo de adaptação.



```
module IntStack = (struct
 type stack = int MyStack.stack
 let empty = MyStack.empty
 let push = MyStack.push
 let pop = MyStack.pop
 let top = MyStack.top
end : sig
 type stack
 val empty : stack
 val push : int → stack → stack
 val pop : stack → stack option
 val top : stack → int option
end)
```

[36] ✓ 0.0s

```
... module IntStack :
 sig
 type stack
 val empty : stack
 val push : int → stack → stack
 val pop : stack → stack option
 val top : stack → int option
 end
```

# Módulos e ficheiros

- A organização em ficheiros separa a estrutura (struct) da assinatura (sig)
- Ficheiros `MyList.ml`, `MyStack.mli`, `MyStack.ml`

```
s > LAP 2024-12 > MyList.ml > ...
type 'a list = Nil | Cons of 'a * 'a list
'a list
let empty = Nil
'a list -> int
let rec length = function
| Nil -> 0
| Cons (_, xs) -> 1 + length xs
'a -> 'a list -> 'a list
let insert x xs = Cons (x, xs)
'a list -> 'a option
let head = function
| Nil -> None
| Cons (x, _) -> Some x
'a list -> 'a list option
let tail = function
| Nil -> None
| Cons (_, xs) -> Some xs
```

```
> LAP 2024-12 > MyStack.mli > ...
type 'a stack
val empty : 'a stack
val push : 'a -> 'a stack -> 'a stack
val pop : 'a stack -> 'a stack option
val top : 'a stack -> 'a option
```

```
s > LAP 2024-12 > MyStack.ml > ...
type 'a stack = 'a MyList.list
'a
let empty = MyList.empty
'a -> 'b -> 'c
let push x xs = MyList.insert
'a
let pop = MyList.tail
'a
let top = MyList.head
```

- Promove a compilação separada.



# Módulos e Funtores (funções de módulos para módulos)



```
module type X = sig
| val x : int
end

module IncX (M : X) = struct
| let x = M.x + 1
end
```

[23]

✓ 0.0s

... module type X = sig val x : int end

... module IncX : functor (M : X) → sig val x : int end

# Módulos e Funtores (funções de módulos para módulos)

```
module type X = sig
 type t
end

module Stack = struct
 module Make (M : X) = struct
 type stack = M.t list
 let empty = []
 let push x xs = x :: xs
 let pop = function
 | [] → None
 | _ :: xs → Some xs
 let top = function
 | [] → None
 | x :: _ → Some x
 end
 end
end

module IntStack = Stack.Make (struct type t = int end)

let s = IntStack.empty
let _ = assert (IntStack.top s = None)
let _ = assert (IntStack.top (IntStack.push 1 s) = Some (1))
let _ = assert (IntStack.pop (IntStack.push 1 s) = Some (IntStack.empty))
```

[71] ✓ 0.0s

# Módulos e Funtores (funções de módulos para módulos)

```
module type OrderedType = sig (* from the standard library *)
 type t
 val compare : t -> t -> int
end
```

```
module Pair = struct
 type t = int * string
 let compare (x1, y1) (x2, y2) =
 if x1 < x2 then -1
 else if x1 > x2 then 1
 else if y1 < y2 then -1
 else if y1 > y2 then 1
 else 0
end
```

```
module PairSet = Set.Make(Pair)
```

```
let s = PairSet.empty
let s = PairSet.add (1, "one") s
let s = PairSet.add (2, "two") s
let s = PairSet.add (3, "three") s
let s = PairSet.add (4, "four") s
let _ = assert (PairSet.mem (2, "two") s)
```



# Módulos e Funtores (funções de módulos para módulos)

```
module StringMap = Map.Make(String) (* from the standard library *)

let m = StringMap.empty
let m = StringMap.add "joao" 30 m
let m = StringMap.add "maria" 25 m
let m = StringMap.add "carlos" 35 m

let _ = assert (StringMap.find "joao" m = 30)
let _ = assert (StringMap.mem "maria" m)
let _ = assert (not (StringMap.mem "pedro" m))
```

# Módulos e Funtores (funções de módulos para módulos)

```
module type OrderedType = sig (* from the standard library *)
 type t
 val compare : t -> t -> int
end
```

```
module Pair = struct
 type t = int * string
 let compare (x1, y1) (x2, y2) =
 if x1 < x2 then -1
 else if x1 > x2 then 1
 else 0
end
```

```
end
```

```
module I
```

```
let s =
```

```
let s =
```

```
let s =
```

```
let s = PairSet.add (3, "three") s
```

```
let s = PairSet.add (4, "four") s
```

```
let _ = assert (PairSet.mem (2, "two") s)
```

```
module PairMap = Map.Make(Pair)
```

```
let m = PairMap.empty
```

```
let m = PairMap.add (1, "a") "first" m
```

```
let m = PairMap.add (2, "b") "second" m
```

```
let _ = assert (PairMap.find (1, "a") m = "first")
```

```
let _ = assert (not (PairMap.mem (3, "c") m))
```



# Mutabilidad

# Mutability

---

- A linguagem OCaml não é uma linguagem *pura*, é possível programar com efeitos laterais.
- Exemplos de efeitos laterais: I/O (printing), leitura e escrita de ficheiros, variáveis de estado, bases de dados, comunicação em geral.
- Mutabilidade permite a implementação de estruturas de dados mais eficientes do que as puras (ex: hash table, doubly linked lists, cyclic graphs)
- A mutabilidade torna a programação mais difícil. Não é fácil raciocinar sobre a mudança de estado (sem enumerar todos os passos).

# Refs!

- Um valor de tipo ref é como um apontador em C, ou uma referência para um objecto ou array em Java.
- As operações de criação e desreferenciação (na heap) são explícitas, como o malloc, ou o “\*” em C.
- Criação e desreferenciação são diferentes da utilização de variáveis de estado (stack) nas linguagens imperativas (C, Java, etc.)

```
let x = ref 0;;
[1] ✓ 0.0s
... val x : int ref = {contents = 0}

!x ;;
[2] ✓ 0.0s
... - : int = 0

x := !x + 1
[3] ✓ 0.0s
... - : unit = ()

▷ ✓ !x ;;
[4] ✓ 0.0s
... - : int = 1
```



# Igualdade “física”

- O operador de igualdade do OCaml (=) testa os valores pela sua estrutura.
- A função Stdlib.(==) testa se duas referências são a mesma. Interessante para algoritmos sobre estruturas de dados dinâmicas (references, arrays, byte sequences, records com campos mutáveis, etc.)



```
let r1 = ref 42
let r2 = ref 42

let _ = assert (not (r1 = r2))
let _ = assert (r1 ≠ r2)
let _ = assert (!r1 = !r2)
let _ = assert (r1 = r2)
```

[35] ✓ 0.0s

```
... val r1 : int ref = {contents = 42}
... val r2 : int ref = {contents = 42}
... - : unit = ()
... - : unit = ()
... - : unit = ()
... - : unit = ()
```

# Aliasing

- Com referências teremos aliasing:  
“Os nomes da stack permitem mais que um caminho para uma mesma célula de memória.”
- Com aliasing o raciocínio fica ainda mais difícil. Para analisar a independência de dois segmentos de código (threads, caller/callee) é preciso eliminar/controlar o aliasing.
- O problema do aliasing em linguagens imperativas é “resolvido” em linguagens como o RUST.



```
let x = ref 42 ;;
let y = ref 42 ;;
let z = x ;;
x := 43 ;;
let w = !y + !z ;;
```

[7]

✓ 0.0s

```
... val x : int ref = {contents = 42}
... val y : int ref = {contents = 42}
... val z : int ref = {contents = 42}
... - : unit = ()
... val w : int = 85
```

# Aliasing

- Com re  
“Os n  
um ca  
memo
- Com  
difícil  
dois s  
caller  
o alia
- O pro  
imper  
como e

```
fn main() {
 let mut x = 42;

 let r1 = &mut x;
 let r2 = &mut x; // COMPILE ERROR: cannot borrow `x`
 // as mutable more than once at a time

 *r1 += 1;
 println!("{}", r2);
}
```

Rust

42}  
42}  
42}



# Aliasing

- Com re  
“Os n  
um ca  
memo
- Com  
difícil  
dois s  
caller  
o alia
- O pro  
imper  
como e

```
fn main() {
 let mut x = 42;
 let z = &x; // shared borrow – z aliases x

 x = 43; // COMPILE ERROR: cannot assign to `x`
 // because it is borrowed

 let w = *z;
 println!("{}", w);
}
```

Rust

42}  
42}  
42}

# Exemplo de um contador

- A função `next_val` retorna um valor diferente cada vez que é chamada. Tem efeitos laterais.
- A criação de uma variável tem que ser separada da função que incrementa.

```
let next_val_broken = fun () →
 let counter = ref 0 in
 incr counter;
 !counter
[14] ✓ 0.0s
... val next_val_broken : unit → int = <fun>

▷ next_val_broken ();;
next_val_broken ();;
[15] ✓ 0.0s
... - : int = 1
... - : int = 1
```

```
▷ let counter = ref 0

let next_val =
 fun () →
 counter := !counter + 1;
 !counter
[10] ✓ 0.0s
... val counter : int ref = {contents = 0}
... val next_val : unit → int = <fun>

▷ next_val ();;
next_val ();;
[11] ✓ 0.0s
... - : int = 1
... - : int = 2
```

# Exemplo de um contador

- A função `next_val` retorna um valor diferente cada vez que é chamada. Tem efeitos laterais.
- A criação de uma variável tem que ser separada da função que incrementa.

```
▷ ~
module Counter1 = Counter.Make();;
Counter1.next_val ();;
Counter1.next_val ();;
module Counter2 = Counter.Make();;
Counter2.next_val ();;
[28] ✓ 0.0s
... module Counter1 :
 sig type t = int ref val counter : int ref val next_val : unit → int end
... - : int = 1
... - : int = 2
... module Counter2 :
 sig type t = int ref val counter : int ref val next_val : unit → int end
... - : int = 1
```

```
▷ ~
module Counter = struct
 module Make() = struct
 type t = int ref
 let counter = ref 0
 let next_val () =
 counter := !counter + 1;
 !counter
 end
 end
end
[25] ✓ 0.0s
... module Counter :
 sig
 module Make :
 functor () →
 sig
 type t = int ref
 val counter : int ref
 val next_val : unit → int
 end
 end
```